

Net-Centric Lab 4

Name: Mohamed Mahmoud Shamsia

ID: 8666

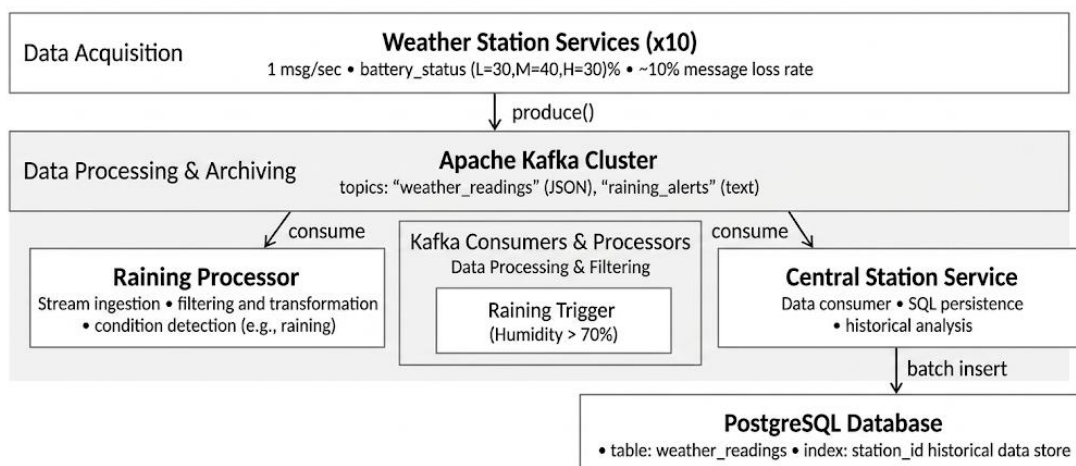
Name: Yassin Mohamed Mady

ID: 8862

1. Introduction:

This report presents the implementation of an end-to-end, stream-processing weather station monitoring system built on a microservices architecture. The primary objective of Lab 4 is to handle high-frequency data streams typical of Internet of Things (IoT) environments. In this system, multiple distributed weather stations act as data sources, generating and publishing weather metrics at regular intervals to Apache Kafka topics. Apache Kafka provides stream ingestion and buffering, while Kafka Processors perform real-time data processing, specifically evaluating humidity metrics to detect and alert on raining conditions when humidity exceeds 70%. A Central Base Station consumes the stream and stores all readings into a relational SQL database using batch inserts for optimized I/O operations. Finally, the entire application stack is containerized with Docker and deployed across a cluster using Kubernetes.

2. System Architecture:



- **Top Layer (Data Acquisition):** 10 Weather Station blocks feeding into a single stream.
- **Middle Layer (Stream Processing):** Apache Kafka Broker containing the weather-raw-data topic, a Kafka Streams processor node filtering for humidity > 70%, and the raining-alerts topic.
- **Bottom Layer (Persistence):** Central Station connected to a Relational SQL Database.

3. Technologies and Prerequisites:

- Docker
- Kubernetes (Minikube, Kind, or cloud-managed Kubernetes)
- Java 17+
- Apache Kafka + Zookeeper
- PostgreSQL or MySQL
- Kafka client libraries

4. Weather Station Mock Implementation

Weather Station Simulation: Each weather station generates and sends one weather-status message every second to the Kafka topic weather-readings. The battery status is randomly assigned with a distribution of 70% high, 20% medium, and 10% low. To simulate network unreliability, there is a 10% chance that a message is dropped and not sent to Kafka.

Required battery distribution:

- Low: 30%
- Medium: 40%
- High: 30%
- Messages randomly dropped: 10%

Weather message fields:

Field	Type	Description
station_id	Long	Weather station identifier
s_no	Long	Auto-incrementing sequence number per station
battery_status	String	low / medium / high
status_timestamp	Long	Unix timestamp
humidity	Integer	Humidity percentage
temperature	Integer	Temperature in Fahrenheit
wind_speed	Integer	Wind speed in km/h

```
mohamed@mohamed-VirtualBox: ~/Lab4-Weather$ kubectl logs deployment/weather-station-1 --tail=20
Station 1 SENT: {"station_id":1,"s_no":435,"weather":{"temperature":87,"humidity":69,"wind_speed":84},"status_timestamp":1787415822,"battery_status":"high"}
Station 1 SENT: {"station_id":1,"s_no":436,"weather":{"temperature":83,"humidity":51,"wind_speed":14},"status_timestamp":1787415824,"battery_status":"medium"}
Station 1 DROPPED message s_no=437
Station 1 SENT: {"station_id":1,"s_no":438,"weather":{"temperature":75,"humidity":61,"wind_speed":50},"status_timestamp":1787415826,"battery_status":"low"}
Station 1 SENT: {"station_id":1,"s_no":439,"weather":{"temperature":96,"humidity":58,"wind_speed":82},"status_timestamp":1787415827,"battery_status":"medium"}
Station 1 SENT: {"station_id":1,"s_no":440,"weather":{"temperature":78,"humidity":78,"wind_speed":27},"status_timestamp":1787415828,"battery_status":"low"}
Station 1 SENT: {"station_id":1,"s_no":441,"weather":{"temperature":55,"humidity":17,"wind_speed":17},"status_timestamp":1787415829,"battery_status":"low"}
Station 1 SENT: {"station_id":1,"s_no":442,"weather":{"temperature":99,"humidity":63,"wind_speed":13},"status_timestamp":1787415830,"battery_status":"medium"}
Station 1 SENT: {"station_id":1,"s_no":443,"weather":{"temperature":53,"humidity":27,"wind_speed":12},"status_timestamp":1787415831,"battery_status":"low"}
Station 1 SENT: {"station_id":1,"s_no":444,"weather":{"temperature":59,"humidity":43,"wind_speed":60},"status_timestamp":1787415832,"battery_status":"medium"}
Station 1 SENT: {"station_id":1,"s_no":445,"weather":{"temperature":78,"humidity":36,"wind_speed":100},"status_timestamp":1787415833,"battery_status":"low"}
Station 1 SENT: {"station_id":1,"s_no":446,"weather":{"temperature":65,"humidity":16,"wind_speed":56},"status_timestamp":1787415834,"battery_status":"low"}
Station 1 DROPPED message s_no=447
Station 1 SENT: {"station_id":1,"s_no":448,"weather":{"temperature":74,"humidity":91,"wind_speed":18},"status_timestamp":1787415837,"battery_status":"high"}
Station 1 SENT: {"station_id":1,"s_no":449,"weather":{"temperature":81,"humidity":80,"wind_speed":3},"status_timestamp":1787415838,"battery_status":"medium"}
Station 1 SENT: {"station_id":1,"s_no":450,"weather":{"temperature":61,"humidity":54,"wind_speed":64},"status_timestamp":1787415839,"battery_status":"high"}
Station 1 SENT: {"station_id":1,"s_no":451,"weather":{"temperature":106,"humidity":4,"wind_speed":19},"status_timestamp":1787415840,"battery_status":"low"}
Station 1 DROPPED message s_no=452
Station 1 SENT: {"station_id":1,"s_no":453,"weather":{"temperature":63,"humidity":71,"wind_speed":84},"status_timestamp":1787415842,"battery_status":"medium"}
Station 1 SENT: {"station_id":1,"s_no":454,"weather":{"temperature":76,"humidity":64,"wind_speed":96},"status_timestamp":1787415843,"battery_status":"low"}
```

```
mohamed@mohamed-VirtualBox: ~/Lab4-Weather$ kubectl logs deployment/weather-station-10 --tail=20
Station 10 SENT: {"station_id":10,"s_no":570,"weather":{"temperature":97,"humidity":67,"wind_speed":34},"status_timestamp":1787415884,"battery_status":"high"}
Station 10 SENT: {"station_id":10,"s_no":571,"weather":{"temperature":102,"humidity":1,"wind_speed":37},"status_timestamp":1787415885,"battery_status":"high"}
Station 10 SENT: {"station_id":10,"s_no":572,"weather":{"temperature":63,"humidity":72,"wind_speed":13},"status_timestamp":1787415886,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":573,"weather":{"temperature":91,"humidity":20,"wind_speed":11},"status_timestamp":1787415887,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":574,"weather":{"temperature":84,"humidity":75,"wind_speed":76},"status_timestamp":1787415888,"battery_status":"low"}
Station 10 SENT: {"station_id":10,"s_no":575,"weather":{"temperature":71,"humidity":24,"wind_speed":26},"status_timestamp":1787415889,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":576,"weather":{"temperature":58,"humidity":46,"wind_speed":8},"status_timestamp":1787415890,"battery_status":"low"}
Station 10 SENT: {"station_id":10,"s_no":577,"weather":{"temperature":65,"humidity":12,"wind_speed":96},"status_timestamp":1787415891,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":578,"weather":{"temperature":106,"humidity":22,"wind_speed":91},"status_timestamp":1787415892,"battery_status":"low"}
Station 10 SENT: {"station_id":10,"s_no":579,"weather":{"temperature":114,"humidity":4,"wind_speed":38},"status_timestamp":1787415893,"battery_status":"low"}
Station 10 SENT: {"station_id":10,"s_no":580,"weather":{"temperature":72,"humidity":63,"wind_speed":57},"status_timestamp":1787415894,"battery_status":"low"}
Station 10 DROPPED message s_no=581
Station 10 SENT: {"station_id":10,"s_no":582,"weather":{"temperature":56,"humidity":92,"wind_speed":91},"status_timestamp":1787415897,"battery_status":"high"}
Station 10 SENT: {"station_id":10,"s_no":583,"weather":{"temperature":57,"humidity":29,"wind_speed":4},"status_timestamp":1787415898,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":584,"weather":{"temperature":93,"humidity":46,"wind_speed":23},"status_timestamp":1787415899,"battery_status":"low"}
Station 10 SENT: {"station_id":10,"s_no":585,"weather":{"temperature":57,"humidity":14,"wind_speed":51},"status_timestamp":1787415900,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":586,"weather":{"temperature":116,"humidity":14,"wind_speed":81},"status_timestamp":1787415901,"battery_status":"low"}
Station 10 DROPPED message s_no=587
Station 10 SENT: {"station_id":10,"s_no":588,"weather":{"temperature":104,"humidity":26,"wind_speed":7},"status_timestamp":1787415903,"battery_status":"medium"}
Station 10 SENT: {"station_id":10,"s_no":589,"weather":{"temperature":74,"humidity":74,"wind_speed":20},"status_timestamp":1787415904,"battery_status":"medium"}
```

5. Kafka Producer Setup:

Kafka Producer Configuration: The weather station uses the Java Kafka Producer API to connect to the Kafka broker through the bootstrap.servers configuration. String serializers are used for both the message key and value. Each weather station creates a JSON weather-status message and publishes it to the Kafka topic weather-readings using a Kafka producer. The station ID is used as the message key, while the weather data is sent as the message value.

```
mohamed@mohamed-VirtualBox:~/Lab4-Weather$ kubectl logs deployment/kafka-processor --tail 30
RAIN ALERT: {"station_id":1,"s_no":813,"humidity":83,"alert":"Rain detected"}
Received station 3 s_no=808 humidity=100
RAIN ALERT: {"station_id":3,"s_no":808,"humidity":100,"alert":"Rain detected"}
Received station 2 s_no=820 humidity=83
RAIN ALERT: {"station_id":2,"s_no":820,"humidity":83,"alert":"Rain detected"}
Received station 5 s_no=877 humidity=6
Received station 7 s_no=886 humidity=75
RAIN ALERT: {"station_id":7,"s_no":886,"humidity":75,"alert":"Rain detected"}
Received station 9 s_no=890 humidity=69
Received station 10 s_no=892 humidity=11
Received station 6 s_no=842 humidity=65
Received station 8 s_no=883 humidity=82
RAIN ALERT: {"station_id":8,"s_no":883,"humidity":82,"alert":"Rain detected"}
Received station 1 s_no=814 humidity=49
Received station 3 s_no=809 humidity=79
RAIN ALERT: {"station_id":3,"s_no":809,"humidity":79,"alert":"Rain detected"}
Received station 2 s_no=821 humidity=73
RAIN ALERT: {"station_id":2,"s_no":821,"humidity":73,"alert":"Rain detected"}
Received station 5 s_no=878 humidity=51
Received station 6 s_no=843 humidity=89
RAIN ALERT: {"station_id":6,"s_no":843,"humidity":89,"alert":"Rain detected"}
Received station 7 s_no=887 humidity=85
RAIN ALERT: {"station_id":7,"s_no":887,"humidity":85,"alert":"Rain detected"}
Received station 9 s_no=891 humidity=94
RAIN ALERT: {"station_id":9,"s_no":891,"humidity":94,"alert":"Rain detected"}
Received station 10 s_no=893 humidity=69
Received station 4 s_no=904 humidity=41
Received station 8 s_no=884 humidity=41
Received station 1 s_no=815 humidity=68
Received station 2 s_no=822 humidity=60
```

6. Raining Trigger / Kafka Processor:

Kafka Processor: The Kafka Processor consumes weather readings from the weather-readings topic and analyzes each message. If the humidity value is greater than 70%, the reading is classified as a raining condition. The processor then creates a rain-alert message and publishes it to a dedicated Kafka topic called rain-alerts. This separates weather alerts from normal weather data and enables other services to subscribe only to rain notifications.

```
mohamed@mohamed-VirtualBox:~/Lab4-Weather$ kubectl exec -it kafka-69dc456dc7-718z9 -- \
> kafka-console-consumer \
> --bootstrap-server localhost:9092 \
> --topic rain-alerts \
> --from-beginning
{"station_id":5,"s_no":8,"humidity":99,"alert":"Rain detected"}
{"station_id":4,"s_no":10,"humidity":89,"alert":"Rain detected"}
{"station_id":4,"s_no":11,"humidity":77,"alert":"Rain detected"}
{"station_id":6,"s_no":10,"humidity":96,"alert":"Rain detected"}
{"station_id":6,"s_no":13,"humidity":71,"alert":"Rain detected"}
{"station_id":6,"s_no":14,"humidity":94,"alert":"Rain detected"}
{"station_id":6,"s_no":16,"humidity":71,"alert":"Rain detected"}
{"station_id":10,"s_no":6,"humidity":92,"alert":"Rain detected"}
{"station_id":7,"s_no":5,"humidity":94,"alert":"Rain detected"}
{"station_id":7,"s_no":6,"humidity":87,"alert":"Rain detected"}
{"station_id":4,"s_no":14,"humidity":91,"alert":"Rain detected"}
{"station_id":4,"s_no":18,"humidity":94,"alert":"Rain detected"}
{"station_id":5,"s_no":15,"humidity":71,"alert":"Rain detected"}
{"station_id":5,"s_no":16,"humidity":95,"alert":"Rain detected"}
{"station_id":5,"s_no":19,"humidity":96,"alert":"Rain detected"}
{"station_id":1,"s_no":15,"humidity":77,"alert":"Rain detected"}
{"station_id":1,"s_no":16,"humidity":85,"alert":"Rain detected"}
{"station_id":4,"s_no":22,"humidity":80,"alert":"Rain detected"}
{"station_id":4,"s_no":24,"humidity":85,"alert":"Rain detected"}
{"station_id":3,"s_no":17,"humidity":97,"alert":"Rain detected"}
{"station_id":5,"s_no":23,"humidity":94,"alert":"Rain detected"}
{"station_id":6,"s_no":22,"humidity":75,"alert":"Rain detected"}
{"station_id":1,"s_no":23,"humidity":82,"alert":"Rain detected"}
```

7. Central Station:

Central Station: The Central Station acts as a Kafka consumer that continuously reads weather readings from the weather-readings topic and can optionally consume rain alerts from the rain-alerts topic. For each received weather message, it extracts the relevant fields and stores them in a PostgreSQL database. To improve database performance and reduce transaction overhead, readings are written using batch inserts instead of individual insert operations. The recommended batch size is 5,000 records, after which the batch is executed and committed to the database. This approach provides efficient and scalable storage of large volumes of weather data.

```
mohamed@mohamed-VirtualBox:~/Lab4-Weather$ kubectl logs deployment/central-station --tail=30
Saved station 9 s_no=2717 humidity=80 battery=high
Saved station 3 s_no=2625 humidity=80 battery=low
LOW BATTERY ALERT: Station 3
Saved station 8 s_no=2702 humidity=56 battery=high
Saved station 2 s_no=2622 humidity=30 battery=medium
Saved station 7 s_no=2705 humidity=18 battery=low
LOW BATTERY ALERT: Station 7
Saved station 5 s_no=2711 humidity=84 battery=low
LOW BATTERY ALERT: Station 5
Saved station 6 s_no=2670 humidity=20 battery=low
LOW BATTERY ALERT: Station 6
Saved station 4 s_no=2730 humidity=21 battery=high
Saved station 10 s_no=2721 humidity=100 battery=medium
Saved station 9 s_no=2718 humidity=49 battery=low
LOW BATTERY ALERT: Station 9
Saved station 1 s_no=2645 humidity=69 battery=low
LOW BATTERY ALERT: Station 1
Saved station 8 s_no=2703 humidity=67 battery=low
LOW BATTERY ALERT: Station 8
Saved station 2 s_no=2623 humidity=95 battery=medium
Saved station 7 s_no=2706 humidity=82 battery=low
LOW BATTERY ALERT: Station 7
Saved station 5 s_no=2712 humidity=36 battery=low
LOW BATTERY ALERT: Station 5
Saved station 4 s_no=2731 humidity=49 battery=medium
Saved station 10 s_no=2722 humidity=38 battery=high
Saved station 9 s_no=2719 humidity=82 battery=high
Saved station 1 s_no=2646 humidity=85 battery=medium
Saved station 3 s_no=2627 humidity=32 battery=medium
Saved station 2 s_no=2624 humidity=45 battery=medium
```


8. SQL Database:

Minimum required database structure:

Column	Type	Purpose
id	BIGSERIAL PRIMARY KEY	Database record identifier
station_id	BIGINT	Station identifier
s_no	BIGINT	Message sequence number
battery_status	VARCHAR(10)	Battery status
Status_timestamp	BIGINT	Weather reading timestamp
humidity	INT	Humidity
temperature	INT	Temperature
wind_speed	INT	Wind speed

```
weather=# \d weather_readings
```

Table "public.weather_readings"					Default
Column	Type	Collation	Nullable		
id	bigint		not null	nextval('weather_readings_id_seq'::regclass)	
station_id	bigint		not null		
s_no	bigint		not null		
battery_status	character varying(10)		not null		
status_timestamp	bigint		not null		
humidity	integer		not null		
temperature	integer		not null		
wind_speed	integer		not null		

```
weather=# SELECT * FROM weather_Readings ORDER BY id DESC LIMIT 20;
```

id	station_id	s_no	battery_status	status_timestamp	humidity	temperature	wind_speed
6493	3	730	medium	1787495723	17	113	4
6492	10	722	medium	1787495723	16	94	60
6491	5	726	high	1787495723	36	110	43
6490	9	724	high	1787495723	42	57	55
6489	7	728	medium	1787495723	58	64	99
6488	4	725	low	1787495723	79	94	31
6487	6	726	low	1787495723	3	73	1
6486	2	733	medium	1787495723	13	103	79
6485	8	723	high	1787495723	0	98	37
6484	1	732	medium	1787495723	58	79	21
6483	3	729	low	1787495722	37	115	53
6482	10	721	medium	1787495722	61	80	65
6481	5	725	high	1787495722	69	53	94
6480	9	723	medium	1787495722	55	116	14
6479	7	727	high	1787495722	10	98	79
6478	4	724	low	1787495722	81	56	95
6477	6	725	low	1787495722	40	71	29
6476	2	732	medium	1787495722	33	58	66
6475	8	722	medium	1787495722	95	107	93
6474	1	731	medium	1787495722	19	108	44

9. Historical Weather Analysis

Include the SQL queries and their results for the required historical analysis.

Battery Status Distribution per Station:

```
weather=# SELECT
    station_id,
    battery_status,
    COUNT(*) AS readings,
    ROUND(
        100.0 * COUNT(*) /
        SUM(COUNT(*)) OVER (PARTITION BY station_id,
        2
        ) AS percentage
FROM weather_readings
GROUP BY station_id, battery_status
ORDER BY station_id, battery_status;
```

station_id	battery_status	readings	percentage
1	high	218	31.37
1	low	205	29.50
1	medium	272	39.14
2	high	203	28.63
2	low	216	30.47
2	medium	290	40.90
3	high	200	28.69
3	low	208	29.84
3	medium	289	41.46
4	high	212	30.55
4	low	201	28.96
4	medium	281	40.49
5	high	205	29.58
5	low	208	30.01
5	medium	280	40.40
6	high	217	30.96
6	low	209	29.81
6	medium	275	39.23
7	high	196	28.12
7	low	215	30.85
7	medium	286	41.03
8	high	198	28.78
8	low	207	30.09
8	medium	283	41.13
9	high	215	30.80
9	low	186	26.65
9	medium	297	42.55
10	high	204	29.78
10	low	199	29.05
10	medium	282	41.17

Dropped Messages per Station:

```
weather=# SELECT
    station_id,
    MAX(s_no) AS expected_messages,
    COUNT(*) AS received_messages,
    MAX(s_no) - COUNT(*) AS dropped_messages,
    ROUND(
        100.0 * (MAX(s_no) - COUNT(*))
        / MAX(s_no),
        2
    ) AS drop_rate_percent
FROM weather_readings
GROUP BY station_id
ORDER BY station_id;
```

station_id	expected_messages	received_messages	dropped_messages	drop_rate_percent
1	841	746	95	11.30
2	845	761	84	9.94
3	839	748	91	10.85
4	835	743	92	11.02
5	836	745	91	10.89
6	835	752	83	9.94
7	839	750	89	10.61
8	835	742	93	11.14
9	834	752	82	9.83
10	833	742	91	10.92

10. Kubernetes Deployment:

The system is deployed on Kubernetes using custom multi-stage Java Dockerfiles for both the Weather Station and Central Station microservices. Database persistence is guaranteed through a 10Gi PersistentVolumeClaim mounted to a single-replica SQL database Deployment exposed internally on port 5432. Event-driven messaging relies on a Bitnami Kafka and Zookeeper service cluster mapping ports 9092 and 2181 for real-time stream processing. System workloads are orchestrated via a 10-replica Weather Station Deployment simulating distributed data generators, alongside a Central Station consumer Deployment configured to ingest streams and execute batch inserts into the database service.

```
mohamed@mohamed-VirtualBox:~/Lab4-Weather$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
central-station-7dfb89f6d5-r9trd	1/1	Running	0	14m
kafka-69dc456dc7-7l8z9	1/1	Running	0	70m
kafka-processor-86dcb47c5b-5jt8f	1/1	Running	0	14m
postgres-58945679b9-mm9p	1/1	Running	0	66m
weather-station-1-66854c8cc7-sjbd7	1/1	Running	0	14m
weather-station-10-744cd66bcf-gjprv	1/1	Running	0	14m
weather-station-2-744f6b7789-vqxxj	1/1	Running	0	14m
weather-station-3-766c6986b-tkm9t	1/1	Running	0	14m
weather-station-4-d9ff99d77-cnd2l	1/1	Running	0	14m
weather-station-5-f4b979c9c-8mcfv	1/1	Running	0	14m
weather-station-6-7b7dbf5dd-fnxm2	1/1	Running	0	14m
weather-station-7-7fd57574dd-j8b9l	1/1	Running	0	14m
weather-station-8-7b45879845-g8lpq	1/1	Running	0	14m
weather-station-9-fcbb99bcd-2vksb	1/1	Running	0	14m
zookeeper-87b46bc6c-dlz55	1/1	Running	0	111m

11. Problems Encountered and Solutions:

Kafka Connection Refusal: Remote weather stations could not reach Kafka due to mismatched listener configurations.

-> Fix: Configured `KAFKA_CFG_ADVERTISED_LISTENERS` to expose separate internal and external broker ports.

Database I/O Bottleneck: Individual INSERT queries for each incoming reading overwhelmed PostgreSQL.

-> Fix: Implemented bulk JDBC batch inserts at 5,000 records to reduce storage overhead.

Missing Output Topic Crash: The Raining Processor failed on startup because the raining-alerts topic did not exist yet.

-> Fix: Added startup initialization via Kafka AdminClient to pre-create topics.

Kubernetes Pod Startup Race Condition: The Central Station pod crashed repeatedly (CrashLoopBackOff) while waiting for Kafka and PostgreSQL to boot up.

-> Fix: Added initContainers in the YAML manifests to block startup until backend services were ready.

JSON Deserialization Errors: Nested telemetry fields caused parsing failures in Kafka Processors.

-> Fix: Enforced structured Jackson JSON DTO models across all Java producers and consumers.

12. Conclusion:

This lab successfully demonstrated the end-to-end design and deployment of a fault-tolerant, real-time IoT stream processing architecture. The system integrated 10 simulated weather stations emitting continuous telemetry, an Apache Kafka messaging backbone, a Kafka Stream Processor for real-time condition detection, and a Central Base Station for batch persistence into a relational SQL database. Finally, the entire application stack was containerized and orchestrated using Kubernetes.

Through this implementation, several key concepts were reinforced:

- **Kafka & Stream Ingestion:** Understood how Kafka decouples data producers from consumers, handling high-frequency message streams efficiently using dedicated topics and partition keys.
- **Real-time Processing:** Learned to utilize Kafka Processors to inspect continuous streams on the fly, filtering and generating real-time alerts (e.g., detecting humidity > 70%) without incurring heavy database lookup overhead.
- **SQL Persistence & Optimization:** Gained hands-on experience in mitigating database I/O bottlenecks by implementing JDBC batch inserts (5,000 records) rather than single-row writes.
- **Kubernetes Orchestration:** Learned how to define declarative YAML manifests to manage pod lifecycles, service discovery, network configurations, and storage persistence across a distributed cluster.